

Gavril Patrick Louis Loyer 16101509
Miguel Velasques Abilio Piola Alves 11807601
Vitor Chinaglia 11912363

VFF-Nav: Autonomous UAV Navigation for Infrastructure Inspection using Virtual Force Fields

Trabalho de conclusão de curso apresentado
ao Departamento de Engenharia de Com-
putação e Sistemas Digitais da Escola Politéc-
nica da Universidade de São Paulo para
obtenção do título de Engenheiro.

Universidade de São Paulo – USP

Escola Politécnica

Departamento de Engenharia de Computação e Sistemas Digitais (PCS)

Supervisor: Prof. Dr. Bruno Albertini

São Paulo, SP

2025

Autorizo a reprodução e divulgação total ou parcial deste trabalho, por qualquer meio convencional ou eletrônico, para fins de estudo e pesquisa, desde que citada a fonte.

Catálogo-na-publicação

Loyer, Gavril

VFF-Nav: Autonomous UAV Navigation for Infrastructure Inspection using Virtual Force Fields / G. Loyer, M. Velasques Abilio Piola Alves, V. Chinaglia -- São Paulo, 2025.
59 p.

Trabalho de Formatura - Escola Politécnica da Universidade de São Paulo. Departamento de Engenharia de Computação e Sistemas Digitais.

1.Drones 2.Virtual Force Field 3.Genetic algorithm 4.Parameter Optimization I.Universidade de São Paulo. Escola Politécnica. Departamento de Engenharia de Computação e Sistemas Digitais II.t. III.Velasques Abilio Piola Alves, Miguel IV.Chinaglia, Vitor

Abstract

This work addresses the challenge of autonomous navigation for Unmanned Aerial Vehicles (UAVs) in complex environments through the development and validation of a Virtual Force Field (VFF) algorithm. The project implements a reactive obstacle avoidance system where obstacles generate repulsive forces and navigation goals generate attractive forces, creating a computationally efficient solution for real-time drone guidance. A core contribution is the development of a sophisticated parameter optimization system using genetic algorithms to automatically tune the force field parameters, ensuring robust performance across diverse scenarios.

The system was developed and validated within a ROS 2 and Gazebo simulation environment integrated with the PX4 autopilot stack. A complementary C++ simulation framework was developed to enable rapid parameter calibration through evolutionary computation, employing population-based optimization with tournament selection and adaptive mutation strategies. The implemented VFF algorithm successfully demonstrates autonomous navigation with obstacle avoidance in simulated environments.

This research provides a foundation for practical applications in infrastructure inspection, safety monitoring, and automated data collection, offering a deterministic and transparent navigation solution that prioritizes computational efficiency and implementation simplicity. The modular architecture and parameter optimization methodology constitute significant contributions to the field of autonomous robotics, with clear pathways for future integration with physical drone platforms.

Keywords: Autonomous Drones, Virtual Force Field, Obstacle Avoidance, Genetic Algorithms, Parameter Optimization, ROS 2, Gazebo Simulation, PX4.

Resumo

Este trabalho aborda o desafio da navegação autônoma de Veículos Aéreos Não Tripulados (VANTs) em ambientes complexos através do desenvolvimento e validação de um algoritmo de Campo de Força Virtual (VFF). O projeto implementa um sistema reativo de evitação de obstáculos onde obstáculos geram forças repulsivas e objetivos de navegação geram forças atrativas, criando uma solução computacionalmente eficiente para guiamento de drones em tempo real. A contribuição central é a criação de um sistema sofisticado de otimização de parâmetros utilizando algoritmos genéticos para ajustar automaticamente os parâmetros do campo de força, garantindo desempenho robusto em diversos cenários.

O sistema foi desenvolvido e validado em um ambiente de simulação ROS 2 e Gazebo integrado com a stack de piloto automático PX4. Uma estrutura de simulação complementar em C++ foi criada para permitir calibração rápida de parâmetros através de computação evolutiva, empregando otimização baseada em população com seleção por torneio e estratégias adaptativas de mutação. O algoritmo VFF implementado demonstra com sucesso navegação autônoma com evitação de obstáculos em ambientes simulados.

Esta pesquisa fornece uma base para aplicações práticas em inspeção de infraestrutura, monitoramento de segurança e coleta automatizada de dados, oferecendo uma solução de navegação determinística e transparente que prioriza eficiência computacional e simplicidade de implementação. A arquitetura modular e a metodologia de otimização de parâmetros representam contribuições significativas para a área de robótica autônoma, com caminhos claros para futura integração com plataformas de drones físicos.

Palavras-chave: Drones Autônomos, Campo de Força Virtual, Evitação de Obstáculos, Algoritmos Genéticos, Otimização de Parâmetros, ROS 2, Simulação Gazebo, PX4.

Contents

Contents	7
1 INTRODUCTION	11
1.1 Objective	11
1.1.1 Global Objective	11
1.1.2 Specific Objective	12
1.2 Justification	12
1.3 Scope and Limitations	13
1.3.1 Requirements Specification	13
1.3.2 Original Project Scope	14
1.3.3 Change of scope	14
1.4 Work Structure	15
2 CONCEPTUAL ASPECTS	17
2.1 Theoretical Foundation	17
2.1.1 Autonomous Navigation and Robotic Control Architectures	17
2.1.2 Unmanned Aerial Vehicle (UAV) Dynamics	18
2.1.3 Force Field	19
2.1.3.1 Related Physics: Electrostatics	19
2.1.3.2 Artificial Potential Field	20
2.1.4 Virtual Force Field	20
2.1.5 Quaternion	22
2.1.5.1 Quaternion Integration and Angular Motion	22
2.1.5.2 Rotating a Vector	23
2.1.5.3 Quaternion Time Evolution	23
2.2 Literature Review	24
2.2.1 The Foundational Paradigm: Artificial Potential Fields	24
2.2.2 From Potential to Direct Forces: The Virtual Force Field	24
2.2.3 Addressing the Local Minima Problem	25
2.2.4 Alternative and Complementary Methodologies	25
2.2.5 The Emergence of Learning-Based Hybrids	25
2.3 State of the Art	26
3 METHOD AND TOOLS	29
3.1 Research Type	29
3.2 Materials and Equipment	29

3.3	Simulation	29
3.4	PX4	30
3.5	C++ Calibration Simulation	30
3.6	Control Model definition	31
3.6.1	Goal Force Calculation	31
3.6.2	Obstacle Force from Sonar Hits	31
3.6.3	Force Saturation and Squashing	32
3.6.4	Turn Bias for Improved Navigation	32
3.6.5	Velocity and Yaw Setpoint Calculation	33
3.7	Parameter Framework	34
3.8	Search Algorithm configurations	35
3.8.1	A* inspired Algorithm	35
3.8.2	Advanced Parameter Calibration via Genetic Algorithm	36
3.8.2.1	Genetic Algorithm Configuration	36
3.8.2.2	Algorithm Implementation	37
3.8.2.3	Search Strategy	37
4	DEVELOPMENT	39
4.1	Calibration Simulation Development	39
4.2	Control Interface Evolution and Integration Challenges	40
4.2.1	Initial Force/Torque Model and Standalone Success	41
4.2.1.1	Fundamental Integration Failure with PX4	41
4.2.1.2	Complex Orientation-Based Interface Attempt	41
4.2.1.3	Associated Calibration Challenges	42
4.2.1.4	Hybrid Manual-Genetic Solution	43
4.2.2	Successful Velocity/Yaw Interface	44
4.2.3	Integration Details	45
4.3	Data Analysis Method	45
5	RESULTS AND DISCUSSION	47
5.1	Introduction	47
5.2	Experimental Results	47
5.2.1	Experimental Setup and Testing Scenarios	47
5.2.2	Quantitative Performance Metrics	48
5.2.3	Qualitative Behavioral Analysis	48
5.2.3.1	Sonar System Performance	48
5.2.3.2	Control Interface Stability	49
5.2.4	Performance Interpretation	49
5.2.5	Comparison with Theoretical Expectations	49
5.2.6	Limitations and Sources of Error	49

5.2.7	Design Choices and Alternatives	50
5.2.8	Significant Findings and Implications	50
6	FINAL CONSIDERATIONS	53
6.1	Summary of Main Results	53
6.2	Conclusions	54
6.3	Work Contributions	54
6.3.1	Technical Contributions	54
6.3.2	Scientific Contributions	54
6.3.3	Educational Contributions	55
6.4	Future Work	55
6.4.1	Algorithm Improvements	55
6.4.1.1	Sensor System Enhancements	55
6.4.1.2	Memory and Mapping Improvements	55
6.4.2	Integration with Physical Platforms	56
6.4.3	Functionality Expansion	56
6.4.4	Validation in Practical Applications	56
6.5	Acknowledgments	56
7	ANNEX	59
7.1	Annex A: Photogrammetry and 3D Reconstruction	59
7.1.1	Structure from Motion (SfM) and Multi-View Stereo (MVS) Algorithms	59
7.1.2	Software for Model Generation	59
	BIBLIOGRAPHY	61

1 Introduction

The use of drones, or Unmanned Aerial Vehicles (UAVs), has become increasingly prevalent in recent decades, driven by advances in onboard sensors, control algorithms, and artificial intelligence. While initially confined to military applications, UAVs are now central to a wide array of civilian applications, including public security, environmental mapping, precision agriculture, and the inspection of critical infrastructure (Conterno, Ivan, 2025; Costa, Rayana Pinto, 2025; Reis, Camila Passos, 2025; Junior, Everson Rodrigues Mesiara Costa, 2025; Lucas Faxina, 2021). This versatility highlights the growing relevance of UAVs in both the scientific field and the industrial sector, with projections of growth in investments and demand in new areas of operation. (Marketsandmarkets, 2025). Within this field, a fundamental challenge in mobile robotics is enabling safe and fully autonomous navigation in complex, unstructured environments. The core of this challenge is achieving robust obstacle avoidance. A navigation system's efficacy is directly tied to its capacity to perceive the environment, predict collisions, and execute avoidance maneuvers in real-time to ensure operational safety and efficiency (Oussama Khatib, 1986; Yucong Lin, 2015; Johann Borenstein, 1991). Numerous approaches address this problem, ranging from classical control techniques to modern, hybrid solutions incorporating machine learning (Kenzo Nonami, Wei Wang, Farid Kendoul, Daisuke Nakazawa, Satoshi Suzuki, 2010; Rejane Cacalvante SÁ, 2012; Saurabh Sarkar, 2007; Ning Wang, Jiyang Dai, Jin Ying, 2021; Tongyao Yang, Fengbao Yang, Dingzhu Li, 2024).

This work addresses the following problem: Are there optimal parameters for the Virtual Force Fields method, and how can they be applied to enable an autonomous drone to follow a trajectory and avoid obstacles? The project aims to develop a low-level control strategy that models obstacles as sources of repulsive forces and goals (e.g. destinations and points of interest) as sources of attractive forces. The primary validation was conducted through simulation, with the objective of creating a reliable system applicable to real-world scenarios such as safety inspections in low-visibility environments and the preliminary data capture for 3D modeling of complex spaces

1.1 Objective

1.1.1 Global Objective

To develop and validate, through simulation, an autonomous navigation system for drones using the virtual force fields method, enabling safe obstacle avoidance.

1.1.2 Specific Objective

To achieve the general objective, the following specific steps were undertaken:

- To research and analyze existing autonomous navigation methods, with a focus on Virtual Force Field and Artificial Potential Field algorithms, to establish a theoretical foundation for the project.
- To model and implement a Virtual Force Field algorithm to be used in the ROS2/Gazebo simulation, creating repulsive forces from obstacles and attractive forces towards destination points and points of interest.
- To fine-tune the parameters of our model using a genetic search algorithm in C++.
- To create simulated testing environments in Gazebo, incorporating various static and dynamic obstacle configurations to rigorously validate the navigation system.
- To integrate the developed control algorithm with the simulated PX4 drone, enabling autonomous flight and obstacle avoidance.

1.2 Justification

The investigation and development of the VFF-Nav project are justified by its significant potential impact across technological, economic, social, and environmental dimensions. This project transcends a purely academic exercise by addressing a concrete, real-world need with tangible applications.

Firstly, this work contributes directly to the field of Computer Engineering by implementing and validating a core robotics algorithm—the Virtual Force Fields method—within a modern framework (ROS 2/Gazebo). It serves as a practical investigation into the challenges of autonomous navigation, such as local minima and real-time processing, providing valuable insights that can inform future research in path planning and low-level drone control. By creating an open and modular system, the project offers a foundational platform that can be extended with more complex sensors (e.g., LIDAR) or integrated with higher-level AI planners, thereby advancing the state of autonomous systems.

Secondly, the ability to perform autonomous inspections in confined or hazardous environments has profound economic implications. Industries such as civil construction (inspecting tunnels, ducts, and bridges), energy (inspecting refineries and power line insulators), and logistics (inventory management in warehouses) rely on inspections that are often costly, time-consuming, and dangerous for human workers. A reliable, autonomous drone system can drastically reduce operational costs, minimize inspection downtime, and

decrease the risks associated with human labor in these environments, thereby improving efficiency and productivity.

Thirdly, this project could be used to enhance worker safety. By deploying drones for tasks in environments with low visibility, structural instability, or exposure to hazardous materials, human operators can remain at a safe distance. This project has the potential to prevent accidents and long-term health issues for inspectors and maintenance personnel. Furthermore, the technology can be applied to public security and disaster response, such as inspecting collapsed structures after an earthquake to locate survivors without putting rescue teams at undue risk.

Finally, autonomous drones can also contribute to environmental monitoring and protection. The system could be adapted for tasks such as monitoring deforestation, tracking wildlife in dense forests, or inspecting remote pipelines for leaks. By enabling more frequent, efficient, and less intrusive data collection, this technology supports better environmental management and conservation efforts.

In summary, the VFF-Nav project is justified by its dual value: it serves as a robust academic study in a key area of robotics while simultaneously developing a versatile technological solution with the potential to generate economic savings, enhance social well-being through improved safety, and contribute to environmental sustainability.

1.3 Scope and Limitations

1.3.1 Requirements Specification

Functional Requirements:

- **Autonomous Navigation:** The drone must be capable of navigating autonomously.
- **Obstacle Avoidance:** The drone must not collide with obstacles.
- **Destination Attainment:** The drone must always reach its final destination.

Non-Functional Requirements:

- **Processing Speed:** route modifications must be fast enough to avoid moving obstacles.
- **Flight Duration:** The drone must follow an optimized route for shorter flight duration.
- **Safety Distance:** The drone should never be closer than a safety limit to an obstacle or point of interest to avoid collisions.

Drone Requirements: Although not within the scope of this project, the drone requires the following functionalities:

- **Sensors:** for collecting data about its own position and state, in addition to data about obstacles and points of interest.
- **GPS:** for determining its position in space and mapping relevant points.

1.3.2 Original Project Scope

During the initial phase of the project, the group decided to develop a drone capable of autonomously performing three-dimensional modeling of various structures, offering the possibility of viewing and interacting with the model on mobile devices. To do this, in addition to the drone with autonomous control for circumnavigating the structure, we would need to develop a mobile application to allow user interaction with the 3D model generated from data processed and transmitted by the drone. During this time, our group split into two fronts: the study of 3D modeling and the construction and control of the drone. Due to the results of this research, our advisor informed us that the project would be too extensive for the scope of an undergraduate thesis. Therefore, we decided to use existing software for drone control, ArduPilot ([ArduPilot Dev Team, 2025](#)), and focus on developing the drone and 3D modeling. Thus, our work focused on developing an autonomous drone with the capability to perform three-dimensional modeling of structures, using ArduPilot as the basis for flight control. A portion of the preliminary work on the 3D model is available in [Appendix 7](#).

1.3.3 Change of scope

However, while preparing to acquire the drone parts, we noticed, thanks to our advisor's help, that their cost would exceed the project's possible budget. Furthermore, it was uncertain whether the parts would arrive early enough for the project's development and testing. Prof. Dr. Albertini then put us in contact with a professor and CITI-USP's researcher, José Otávio Brochado Colombini, who had an ongoing project in an area related to our previous one: implementing a virtual force fields method for controlling autonomous drones. Following this change, this work is now delimited by the following scope and limitations:

- The primary validation of the Virtual Force Field algorithm was conducted entirely within a simulated environment using ROS 2 and Gazebo.
- The project focuses on the development and testing of the navigation algorithm. Integration with physical hardware at CITI-USP is considered a future step, contingent upon time and resource availability.

- The implementation of photogrammetry and 3D modeling, which was part of the original project concept, is beyond the scope of this current work.
- The system is designed for structured or semi-structured environments and may have limitations in highly dynamic, unpredictable scenarios.

1.4 Work Structure

This work is organized as follows: Chapter 2 (Conceptual Aspects) presents the theoretical foundation, literature review, and state of the art relevant to autonomous navigation and Virtual Force Fields. Chapter 3 (Method and Tools) details the methodology, including the simulation setup, development process, and experimental procedures. The subsequent chapters present the results obtained, discuss their implications, and finally draw conclusions and suggest directions for future work.

2 Conceptual Aspects

This chapter establishes the conceptual and theoretical groundwork for the VFF-Nav project. It begins by presenting the fundamental theories of autonomous navigation and robotics that underpin our work. Following this, a comprehensive review of the existing literature traces the evolution of obstacle avoidance methods, culminating in an analysis of the current state of the art. The purpose of this chapter is to provide the necessary context to understand both the historical development and the contemporary landscape of autonomous drone navigation, thereby justifying the technical choices made in this project.

2.1 Theoretical Foundation

2.1.1 Autonomous Navigation and Robotic Control Architectures

The control architecture for autonomous navigation in a robot is a continuous loop of real-time data exchange between the robot's software and its actuators (e.g., the components that enable the robot's movement). This continuous loop is based on the Sense-Plan-Act paradigm ([Rodney A. Brooks, 1985](#)): The robot starts by using its different sensors to capture information about its surroundings, learning how the world it occupies is constructed. That is the **Sense** phase. In this project, the UAV will use a GPS to know its position in the environment relative to the goal position and its starting point, and a camera/depth sensor to detect obstacles and potential points of interest.

The robot then analyzes the data received from its sensors to decide on its next actions. This is the **Plan** phase. In this project, the robot will take into account the distance to its goal, using the GPS, and the distance to potential obstacles, using a RADAR.

The robot, based on the actions decided during the plan phase, sends commands to its actuators to initiate the desired motion and thus **Act**. For this project, these actions will be to go straight to the goal, to visit a point of interest, or to evade an obstacle.

However, for this loop to be effective, the robot must determine its position and orientation, a challenge often referred to as the "where am I" problem. There are two major ways to locate oneself: a global way, like using a GPS to get a position on a map, or a more local one, where movement is calculated from a starting point. The problem lies particularly in the uncertainty inherent in both methods. For the global part, environmental noise can affect sensors (e.g., low GPS accessibility), while in a local approach, small, unpredicted drifts can accumulate during the robot's movement. For

this reason, it is beneficial to combine these two methods. This modern solution is called SLAM (Simultaneous Localization and Mapping) ([Leutenegger et al. , 2013](#)).

From this theory, two main autonomous navigation systems have emerged ([Oussama Khatib, 1986](#)): the **Path Planning** and the **Reactive Obstacle Avoidance** approaches. In the Path Planning approach, given a pre-existing map of the environment, the robot computes an optimal path from its starting point to its goal, determining the robot's required movements at each step before it even begins. This approach enables smoother movement as it does not need to analyze sensor information in real-time to choose its next action. However, the robot will be unable to detect moving obstacles and is overly dependent on a potentially outdated map. For the Reactive Obstacle Avoidance approach, the robot uses real-time information from its sensors to recalculate an immediate path to evade obstacles. There is little to no global path planning in this method. However, this method also has issues. The robot requires significant computational power to analyze all the data and make decisions in a timely manner, making swift and quick movement more challenging. Moreover, the robot can become stuck due to conflicting information on how it needs to move, a situation known as local minima. The Virtual Force Field method used in this paper is one type of Reactive Obstacle Avoidance.

2.1.2 Unmanned Aerial Vehicle (UAV) Dynamics

The most common UAV, and the model used for testing in this project, is the quadcopter. It is a drone that uses four "arms," each equipped with one motor. Quadcopters move by adjusting the balance between motor speeds: By changing the spin of all four rotors, it is possible to change the current thrust and altitude of the drone. To move forward (resp. backward) the front rotors are slowed down (resp. sped up) and the rear ones are sped up (resp. slowed down). For left or right movement, the rotors on one side speed up while those on the opposite side slow down (e.g., speeding up the right rotors to go left and vice versa). Finally, speeding up rotors diagonally creates a torque and thus makes the drone rotate (change the yaw).

This complex process makes real-time calculation of movements quite complicated. For this reason, the simulation's drone will be equipped with a low-level flight controller that will handle rotor speeds based on simple commands like "go forward" or "turn left."

To create a more manageable means of control, two distinct control layers were established: the **Low-Level Control**, such as the flight controller, which keeps the drone stable during flight by counteracting disturbances, and the **High-Level Control**, typically the navigation algorithm, which handles tasks like obstacle avoidance. To distinguish between the two, the UAV uses different control frequencies for each method.

2.1.3 Force Field

2.1.3.1 Related Physics: Electrostatics

To understand what a Virtual Force Field is, it is helpful first to understand electrostatics. This topic is relevant because this obstacle avoidance method appears to have been created in a manner analogous to electrostatics. However, it is important to emphasize that this is only an analogy; the robot does not use any electrical charge.

Electrostatics is a branch of electromagnetism that deals with stationary electric forces. The most fundamental law in this area is Coulomb's law, which states that the force between two charged points is directly proportional to the product of their charges and inversely proportional to the square of the distance between them.

The equation is given by:

$$\vec{F} = k_e \frac{q_1 q_2}{r^2} \hat{r} \quad (2.1)$$

Where:

- $\vec{F}$ is the force vector between the charges
- q_1 and q_2 are the magnitudes of the two charges
- r is the distance between them
- $\hat{r}$ is the unit vector in the direction from one charge to the other

However, Coulomb's law is not the only concept of interest; the more powerful concept of the **Electric Field** ($\vec{E}$) is more relevant to this work. The electric field is a way to model the force that a particular charge exerts on its surrounding environment, helping to model the force that every other charged particle will experience.

The electric field created by a charged particle is:

$$\vec{E} = k_e \frac{q}{r^2} \hat{r} \quad (2.2)$$

Note that this equation is the Coulomb force for the case where the second particle's charge is equal to 1. Following this observation, the force on a particular charged particle, with charge q_2 , inside this electric field will be:

$$\vec{F} = q_2 \vec{E} \quad (2.3)$$

One more key principle of electric fields that interests us is the **principle of superposition**. This principle states that in linear systems, which represent a significant portion of natural phenomena, the total force applied to a charged particle present in several electric fields is the sum of the forces induced by each field individually.

Consider fields $\vec{E}_1$, $\vec{E}_2$, and $\vec{E}_3$, with respective charges q_1 , q_2 , and q_3 . The force applied to a charged particle of charge q' will be:

$$\vec{F}_{total}(q') = \vec{F}_{E_1}(q') + \vec{F}_{E_2}(q') + \vec{F}_{E_3}(q') \quad (2.4)$$

$$\vec{F}_{total}(q') = q'(\vec{E}_1 + \vec{E}_2 + \vec{E}_3) \quad (2.5)$$

2.1.3.2 Artificial Potential Field

The Artificial Potential Field (APF) method, introduced by Khatib in ([Oussama Khatib, 1986](#)), is a foundational approach to robot path planning and obstacle avoidance. It is a global, mathematically defined technique that models the entire environment as a scalar potential energy field. In this model, the robot is treated as being inside a virtually created force field induced by the goal and the obstacles. The goal creates a force field that attracts the robot, and each obstacle individually is a source of a repulsive potential, each depending on the distance between the obstacle and the robot, generally in an inverse-square relationship. The mathematical formulation of the total virtual potential $U_{total}(\vec{q})$ that the robot experiences at point $\vec{q}$ in space, using the principle of superposition, is the sum of the different artificially generated potentials:

$$U_{total}(\vec{q}) = U_{attr}(\vec{q}) + U_{rep}(\vec{q}) \quad (2.6)$$

Where:

- $U_{attr}(\vec{q}) = U_{goal}(\vec{q})$
- $U_{rep}(\vec{q}) = \sum_{i=0}^n U_{obstacle_i}(\vec{q})$ with n = number of obstacles

This technique defines a potential field everywhere in the environment before the robot even starts moving. However, this method is prone to the local minima problem, where the attractive and repulsive forces cancel each other out, preventing the robot from ever reaching the goal, or even moving, without external intervention. It also cannot account for obstacles that were not known initially.

2.1.4 Virtual Force Field

The Virtual Force Field (VFF) method is a derived, simplified, and more reactive version of the APF concept. It retains the core "force" analogy but dispenses with the explicit calculation of a global potential field, making it a reactive and local strategy. Effectively, the VFF computes, at each cycle of the SPA paradigm, the force vectors directly based on the robot's sensor readings. This way, the resultant force is less computationally

demanding than the gradient calculation in the APF method, as it is a direct vector sum of the attractive force and the repulsive forces:

$$\vec{F}_{total} = \vec{F}_{attr} + \sum_{i=0}^n \vec{F}_{rep_i} \quad (2.7)$$

Where:

- $\vec{F}_{attr}$ is the attractive force from the goal.
- $\vec{F}_{rep_i}$ is the repulsive force from the i^{th} obstacle (e.g., $\vec{F}_{rep_i} = \frac{k_{rep}}{r^2} \hat{r}$ if the obstacle is at distance r in the $\hat{r}$ direction).

In this way, the robot becomes reactive to its immediate environment. The implementation is also easier.

Feature	Artificial Potential Field (APF)	Virtual Force Field (VFF)
Foundation	Scalar Potential Field $U(\vec{q})$	Direct Force Vectors $\vec{F}$
Calculation	Computes force as $\vec{F} = -\nabla U$	Direct vector sum of forces: $\vec{F}_{total} = \vec{F}_{att} + \sum \vec{F}_{rep,i}$
Scope	Global, analytical	Local, reactive
Computational Load	Higher (requires gradient computation)	Lower (direct vector math)
Implementation	Complex, requires potential field definition	Simple, direct force calculation
Primary Reference	(Oussama Khatib, 1986)	(Saurabh Sarkar, 2007)

Table 1 – Key Differences Between Artificial Potential Field (APF) and Virtual Force Field (VFF) Methods

For the VFF-Nav project, the Virtual Force Field method was selected over the classical Artificial Potential Field approach. While inspired by the same physical principles, the VFF’s direct force calculation provides the computational efficiency and reactive performance necessary for real-time drone navigation in dynamic environments. This derived version maintains the intuitive force-based navigation of APF while being more practical for implementation on embedded systems with limited processing resources.

It is important to emphasize that the explanation of electric fields and electrostatics serves only as a historical and conceptual basis for this model and is strictly an analogy. While this analogy provides a powerful and intuitive physical model for understanding the Virtual Force Field method, it is crucial to note the distinctions in implementation. The robotic system does not generate actual electromagnetic fields. The ‘forces’ are analogously constructed and calculated from sensor data. Furthermore, for practical stability and to

prevent excessively high forces at very small distances, the implementation often uses alternative functions for force decay, such as an inverse-proportional ($\frac{1}{r}$) relationship or defined dead zones, rather than a pure inverse-square ($\frac{1}{r^2}$) law. The value of the analogy lies in its conceptual framework for modeling action-at-a-distance navigation.

Electromagnetic Concept	Mathematical Expression	Virtual Force Field Equivalent
Coulomb's Law	$\vec{F} = k_e \frac{q_1 q_2}{r^2} \hat{r}$	Repulsive Force: $\vec{F}_{rep} = k_{rep} \frac{1}{r^2} \hat{r}_{away}$ Attractive Force: $\vec{F}_{att} = k_{att} \frac{1}{r^2} \hat{r}_{toward}$
Electric Field ($\vec{E}$)	$\vec{E} = k_e \frac{q}{r^2} \hat{r}$	The "Virtual Force Field" itself - A vector field defined at every point in space
Force on a Test Charge	$\vec{F} = q_{test} \vec{E}$	The drone calculates the net force by summing the field vectors at its location
Superposition Principle	$\vec{E}_{total} = \vec{E}_1 + \vec{E}_2 + \dots + \vec{E}_n$	$\vec{F}_{total} = \vec{F}_{att} + \vec{F}_{rep1} + \vec{F}_{rep2} + \dots$

Table 2 – Mapping Between Electromagnetic Concepts and Virtual Force Field Implementation

2.1.5 Quaternion

In this work, quaternions are used to efficiently represent 3D rotations and, in this way, help us calculate the torque (rotation) of our drone to control where the different sensors are facing.

2.1.5.1 Quaternion Integration and Angular Motion

Quaternions are used to represent the drone's orientation in 3D space without running the risk of encountering a gimbal lock, where rotation axes become aligned and limit possible rotations.

A quaternion is a four-dimensional number represented by a real part and a 3D imaginary part:

$$q = (w, x, y, z) \quad (2.8)$$

$$q = w + xi + yj + zk \quad (2.9)$$

However, as humans, we cannot perceive this fourth dimension, so, to resolve this issue, we will only be working with unit quaternions, i.e., quaternions that have a distance of 1 from the origin. In this way, we can actually represent the quaternions in a 3D plane using a stereographic projection. Stereographic projection provides a method for visualizing unit quaternions in three dimensions (3blue1brown, 2018).

The unit quaternion, for representing a rotation of θ radians around the axis $\vec{u}$, can also be written as:

$$q = \left[\cos\left(\frac{\theta}{2}\right), \vec{u} \sin\left(\frac{\theta}{2}\right) \right] \quad (2.10)$$

2.1.5.2 Rotating a Vector

To rotate a vector $\vec{v}$ by a quaternion $\mathbf{q}$, we first need a quaternion representation of $\vec{v}$, defined as $\mathbf{p} = [0, \vec{v}]$. Now, the rotated vector can be calculated:

$$[w_{rot}, \vec{v}'] = qpq^{-1} \quad (2.11)$$

For two quaternions $(a_1, \vec{b}_1)$ and $(a_2, \vec{b}_2)$, the multiplication (Hamilton product) follows:

$$(a_1, \vec{b}_1) (a_2, \vec{b}_2) = (a_1 a_2 - \vec{b}_1 \cdot \vec{b}_2, a_1 \vec{b}_2 + a_2 \vec{b}_1 + \vec{b}_1 \times \vec{b}_2) \quad (2.12)$$

This rule defines how scalar and vector parts interact. The dot (.) products generate scalars, while cross products handle the rotation direction.

2.1.5.3 Quaternion Time Evolution

Given the body's angular velocity $w = (w_x, w_y, w_z)$, the time derivative of the orientation quaternion is:

$$\dot{q} = \frac{1}{2} [0, w] q \quad (2.13)$$

This means the quaternion's rate of change is half the quaternion product between $\mathbf{q}$ and the angular-velocity quaternion.

Expanding component-wise gives:

$$\dot{q} = \frac{1}{2} \begin{bmatrix} 0 & -w_x & -w_y & -w_z \\ w_x & 0 & w_z & -w_y \\ w_y & -w_z & 0 & w_x \\ w_z & w_y & -w_x & 0 \end{bmatrix} \begin{bmatrix} w & x & y & z \end{bmatrix} \quad (2.14)$$

The matrix in the equation above, often denoted $\Omega(w)$, is simply a compact way to represent all the sign changes that naturally appear in the Hamilton product when multiplying by $[0, w]$. It doesn't introduce new math, it is just a shorthand that expands

to:

$$[0, w] q = (-w_x x - w_y y - w_z z, w w_x + w_y z - w_z y; w w_y + w_z x - w_x z; w w_z + w_x y - w_y x) \quad (2.15)$$

This is exactly the same as performing the quaternion multiplication manually; the matrix form simply makes it linear.

2.2 Literature Review

Autonomous navigation for drones is a cornerstone of modern mobile robotics research. A critical component of this autonomy is robust obstacle avoidance, which has been addressed through a spectrum of methodologies, ranging from classical control strategies to contemporary artificial intelligence-based solutions. This review synthesizes the existing literature, tracing the evolution of these approaches and establishing the theoretical foundation for the present work.

2.2.1 The Foundational Paradigm: Artificial Potential Fields

The conceptual groundwork for reactive navigation was firmly established by (Oussama Khatib, 1986) with the introduction of the Artificial Potential Field (APF) method. This approach models the robot's environment as a scalar field of potential energy, where the goal generates an attractive potential and obstacles generate repulsive potentials. The robot then moves in the direction of the steepest descent of this combined field, effectively being drawn to the goal while being repelled from obstacles. The elegance of this method lies in its mathematical simplicity and intuitive physical analogy. However, a significant and well-documented limitation is its susceptibility to local minima—points in space where the attractive and repulsive forces balance, trapping the robot (Ning Wang, Jiyang Dai, Jin Ying, 2021).

2.2.2 From Potential to Direct Forces: The Virtual Force Field

To streamline the APF concept for real-time performance, the Virtual Force Field (VFF) method emerged as a practical derivation. Rather than computing a full potential field and its gradient, VFF directly calculates force vectors based on immediate sensor data. As demonstrated by (Saurabh Sarkar, 2007), this approach maintains the core intuition of attractive and repulsive interactions but implements them through direct vector summation, significantly reducing computational overhead. This makes VFF particularly suitable for systems with limited processing resources, offering a robust solution for local, reactive obstacle avoidance. The method's efficacy has been validated on various platforms, including ground vehicles, as shown by (B.Mashadia, M.A.Vesala, H.Amania, 2017).

2.2.3 Addressing the Local Minima Problem

The issue of local minima has been a central focus of subsequent research. For ground robots in 2D environments, (Johann Borenstein, 1991) proposed the Vector Field Histogram (VFH), which provided a fast and effective solution by integrating data over time and space to identify navigable directions. Extending these principles to the more complex domain of UAVs, (Ning Wang, Jiyang Dai, Jin Ying, 2021) developed an improved APF combined with consensus theory, enabling drone formations to navigate while avoiding inter-collisions and escaping local minima through cooperative behavior.

2.2.4 Alternative and Complementary Methodologies

Parallel to the development of field-based methods, other strategic approaches have been explored. (Yucong Lin, 2015) investigated a method based on reachable sets for collision avoidance, which is particularly powerful for predicting and avoiding dynamic obstacles. This represents a more planning-oriented approach compared to the purely reactive nature of basic VFF.

2.2.5 The Emergence of Learning-Based Hybrids

A significant recent trend has been the integration of classical methods with machine learning. (Tongyao Yang, Fengbao Yang, Dingzhu Li, 2024) exemplifies this direction, proposing a multi-strategy path planning framework that leverages Large Language Models (LLMs) to generate sophisticated reward functions. This hybrid approach seeks to overcome the limitations of both purely reactive systems and those requiring extensive pre-mapping, aiming for robust performance in high-speed, high-density environments.

In summary, the scholarly landscape reveals a clear trajectory from the theoretical elegance of potential fields (Oussama Khatib, 1986) to their practical implementation as Virtual Force Fields (Saurabh Sarkar, 2007), followed by continuous refinements to address inherent limitations such as local minima (Ning Wang, Jiyang Dai, Jin Ying, 2021). The ongoing integration of data-driven techniques points toward increasingly adaptive and intelligent systems (Tongyao Yang, Fengbao Yang, Dingzhu Li, 2024).

Within this context, the VFF-Nav project aligns itself with the proven and computationally efficient paradigm of Virtual Force Fields. This choice is strategically justified for a proof-of-concept implementation, as it provides a deterministic, transparent, and effective framework for achieving core autonomous navigation and obstacle avoidance, forming a solid foundation upon which more complex, learning-based capabilities could be integrated in the future.

2.3 State of the Art

This section examines the most advanced current solutions in both academic research and commercial applications for autonomous drone navigation and obstacle avoidance. The field has evolved significantly from classical approaches to incorporate sophisticated artificial intelligence techniques, each with distinct advantages and limitations.

Method / Technology	Core Principle	Pros	Cons	Key Example / Reference
Classical VFF/APF	Attractive (goal) and repulsive (obstacle) force vectors	High intuitiveness; Computationally efficient; Guarantees real-time performance	Prone to local minima ; Performance depends heavily on gain tuning; Limited in complex environments	Foundational work by (Oussama Khatib, 1986); implementations like (Saurabh Sarkar, 2007)
Improved 3D APF	Extends APF with 3D dynamics and consensus for swarms	Solves key 3D local minima problem; Enables coordinated swarm flight	Increased complexity; Still largely reactive without long-term planning	(Ning Wang, Jiyang Dai, Jin Ying, 2021)
Vision-Based SLAM/VIO	Uses camera(s) + IMU to simultaneously map environment and localize	GPS-denied operation; Rich environmental understanding; Creates reusable maps	Computationally intensive; Performance depends on lighting/texture; Scale estimation issues (monocular)	(Mur-Artal, Raúl and Montiel, J. M. M. and Tardós, Juan D., 2015); (Qin, Tong and Li, Peiliang and Shen, Shaojie, 2018)
LiDAR-based Navigation	Uses laser scanning for precise 3D mapping and obstacle detection	High accuracy; Works in darkness; Direct 3D measurements	High cost, weight, and power consumption; Limited range in adverse weather	(DJI Enterprise, 2023)
Learning-Based Methods (RL/LLM)	Uses neural networks to learn navigation policies from data	Adaptive to new environments; Can handle complex scenarios; No manual parameter tuning	Large data requirements; Unpredictable failures; "Black box" decisions	(Tongyao Yang, Fengbao Yang, Dingzhu Li, 2024)
Reachability Analysis	Computes mathematically guaranteed safe trajectories	Provable safety guarantees; Excellent for dynamic obstacles	Computationally expensive; Conservative trajectories	(Yucong Lin, 2015)

Table 3 – Comparative Analysis of State-of-the-Art Obstacle Avoidance Methods for UAVs

In the commercial sector, companies like DJI and Skydio have implemented advanced obstacle avoidance systems that typically combine multiple sensors:

- **Skydio's Visual AI:** Uses a sophisticated array of cameras and neural networks to create 3D maps of the environment in real-time, allowing for exceptionally robust obstacle avoidance in complex natural environments. ([Skydio, 2024](#))
- **DJI's O3+ System:** Combines visual, infrared, and ultrasonic sensors with advanced path planning algorithms, offering comprehensive all-directional avoidance for enterprise-grade drones. ([DJI Enterprise, 2023](#))

However, the analysis of current state-of-the-art solutions reveals several persistent challenges:

- **Computational Efficiency vs. Performance Trade-off :** Lightweight methods like classical VFF struggle with local minima and complex environments, while more robust solutions (SLAM, learning-based) require substantial computational resources unsuitable for low-cost platforms.
- **Predictability and Safety :** Learning-based methods, while powerful, often operate as "black boxes" with unpredictable failure modes, making them unsuitable for safety-critical applications without extensive verification.
- **Generalization Limitations :** Most systems are optimized for specific environmental conditions (e.g., outdoor with good lighting, structured environments) and struggle with domain shift.
- **Cost-Accessibility Gap :** High-performance systems incorporating LiDAR or advanced AI processors remain prohibitively expensive for many applications.

3 Method and Tools

3.1 Research Type

This work is characterized as an **experimental development** project with a strong component of **simulation**. The objective is not merely to observe phenomena but to create, implement, and validate a functional system (the autonomous navigation algorithm) within a controlled, simulated environment. The approach is primarily qualitative, focusing on the implementation and behavioral analysis of the control system, but incorporates quantitative metrics for performance evaluation.

3.2 Materials and Equipment

The project was developed entirely within a simulation environment. The primary tools and their versions are listed below.

- **Operating System:** Ubuntu 24.04.3 LTS
- **Middleware/Framework:** ROS 2 (Robot Operating System 2) - Jazzy Jalisco
- **Simulator:** Gazebo (Version 11)
- **Flight Stack:** PX4 Autopilot (Version 1.14)
- **Programming Language:** Python 3.10
- **Computer:** Personal computers with minimum 8GB RAM and a multi-core processor.

3.3 Simulation

A simulated drone was essential for developing and testing the solution. On the one hand, it was also necessary to have a calibration simulation to search for an optimal set of parameters for the Virtual Force Field algorithm. This calibration was done using C++ and will be discussed more in depth in the section [3.5](#).

On the other hand, a validation simulation is required to ensure the validity of the found parameters. ROS 2, PX4 and Gazebo were selected as the primary tools for this purpose. Gazebo is a simulation platform used to create a virtual world in which different robotics projects can be tested. It will be used to create worlds with different

sets of obstacles to variably test the algorithm for autonomous drone flight. ROS 2, or Robot Operating System 2, is software for creating robot applications, and in this project it is used to implement the force field algorithm and to manage the communication of sensor data and control signals to and from the drone. Within ROS 2, one can use C++ or Python with subscriber and publisher nodes to send and receive data from the robot. In the project, the drone will send its current state and receive information about the environment to analyze and respond to the presence of obstacles and its goal.

3.4 PX4

The PX4 drone was chosen for testing within ROS 2 due to its seamless integration with the framework and its comprehensive sensor suite, which includes a low-level flight controller that simplifies movement computation. Furthermore, this model is similar to the model available for the project at CITI-USP.

3.5 C++ Calibration Simulation

To calibrate the parameters of the Virtual Force Field (VFF) algorithm, a custom C++ simulation was developed with the primary objective of executing a large volume of simulated flights—estimated on the order of millions—efficiently, enabling systematic exploration of the parameter space.

The choice of a dedicated C++ implementation is justified by the need for high computational performance, since the integrated ROS 2 and Gazebo simulation environment entails significantly longer execution times, making large-scale evaluation impractical. Furthermore, the custom simulation eliminates the complexity associated with the numerous configurable parameters of the ROS/Gazebo environment that are irrelevant to the calibration process, focusing exclusively on the force field navigation model.

Two search algorithms were implemented and evaluated on this platform: an A*-inspired approach adapted for continuous spaces, and a sophisticated genetic algorithm (GA) designed for multivariate optimization, detailed in Section 3.8.

3.6 Control Model definition

3.6.1 Goal Force Calculation

The goal generates an attractive force field. Let:

$$\begin{aligned}\mathbf{r}_g &= \mathbf{g} - \mathbf{p}_d \quad (\text{vector from drone to goal}) \\ d_g &= \|\mathbf{r}_g\| \quad (\text{distance to goal}) \\ \hat{\mathbf{r}}_g &= \begin{cases} \frac{\mathbf{r}_g}{d_g}, & d_g > 0 \\ 0, & d_g = 0 \end{cases}\end{aligned}$$

With parameters:

- q_{goal} : Goal charge (always negative)
- $f_{\text{goal_const}}$: Constant attractive term (≥ 0)
- $\varepsilon_{\text{goal}}$: Softening parameter to prevent singularity

The goal contribution is:

$$E_{\text{goal}} = \frac{q_{\text{goal}}}{d_g^2 + \varepsilon_{\text{goal}}^2}, \quad \mathbf{F}_{\text{goal}} = (E_{\text{goal}} - f_{\text{goal_const}})\hat{\mathbf{r}}_g \quad (3.1)$$

Since $q_{\text{goal}} < 0$ and $f_{\text{goal_const}} \geq 0$, $\mathbf{F}_{\text{goal}}$ is always attractive (points from drone toward goal). The $\varepsilon_{\text{goal}}$ parameter prevents numerical instability when $d_g \rightarrow 0$.

3.6.2 Obstacle Force from Sonar Hits

For each sonar hit h_i , compute:

$$\begin{aligned}\mathbf{r}_i &= \mathbf{p}_d - h_i \quad (\text{vector from hit to drone}) \\ d_i &= \|\mathbf{r}_i\| \\ \hat{\mathbf{r}}_i &= \frac{\mathbf{r}_i}{d_i}\end{aligned}$$

The charge for each hit depends on sonar type:

$$q_i = \begin{cases} q_{\text{plane}}, & h_i \in \mathcal{H}_{\text{plane}} \\ q_{\text{vert}}, & h_i \in \mathcal{H}_{\text{vert}} \end{cases} \quad (3.2)$$

Where $q_{\text{plane}} = 1.0$ (fixed) and q_{vert} is a tunable parameter.

Each hit corresponds to an artificial field present in the calculation of forces on the drone:

$$E_i = \frac{q_i}{d_i^2 + \varepsilon_{\text{obs}}^2}, \quad \mathbf{e}_i = E_i \hat{\mathbf{r}}_i \quad (3.3)$$

Alternatively, it was also tested the usage of a different field function

$$E_i = \frac{q_i}{1 - \exp(K_{sharp} \cdot (d_i - d_{intensity}))}, \quad \mathbf{e}_i = E_i \hat{\mathbf{r}}_i, \quad (3.4)$$

This second equation is based on the principle that if the drone is far enough from an obstacle (beyond $d_{intensity}$), it should not be affected by it. Conversely, if the drone gets too close, the repulsive force intensity should increase sharply, controlled by the parameter K_{sharp}

The raw obstacle field is the vector sum:

$$\mathbf{F}_{\text{obs,raw}} = \sum_{h_i \in \mathcal{H}_{\text{plane}}} \mathbf{e}_i + \sum_{h_j \in \mathcal{H}_{\text{vert}}} \mathbf{e}_j \quad (3.5)$$

3.6.3 Force Saturation and Squashing

To prevent excessively large forces near obstacles, $\mathbf{F}_{\text{obs,raw}}$ passes through a smooth saturation function:

Let $f = \|\mathbf{F}_{\text{obs,raw}}\|$ and $x = f/f_{\text{sat}}$, where f_{sat} is the saturation scale parameter.

If $f = 0$, then $\mathbf{F}_{\text{obs}} = 0$.

Otherwise:

$$\mathbf{F}_{\text{obs}} = \begin{cases} \frac{f_{\text{fac,squash}}}{f_{\text{sat}}} \mathbf{F}_{\text{obs,raw}}, & x \leq 0.3 \\ f_{\text{fac,squash}} \cdot \tanh(x) \cdot \frac{\mathbf{F}_{\text{obs,raw}}}{f}, & x > 0.3 \end{cases} \quad (3.6)$$

This saturation ensures bounded obstacle influence regardless of the number of detected obstacles or sonar ray density.

3.6.4 Turn Bias for Improved Navigation

To encourage the drone to circumnavigate obstacles rather than pushing directly against them, a turn bias parameter $\alpha \in [0, 1]$ is applied:

1. Compute magnitudes:

$$\begin{aligned} f_{\text{goal}} &= \|\mathbf{F}_{\text{goal}}\| \\ f_{\text{obs}} &= \|\mathbf{F}_{\text{obs}}\| \end{aligned}$$

2. Obstacle importance ratio:

$$r = \text{clamp}\left(\frac{f_{\text{obs}}}{f_{\text{goal}} + \epsilon}, 0, 1\right) \quad (3.7)$$

3. Effective bias: $\alpha_{\text{eff}} = \alpha \cdot r$

4. Decompose guidance field:

$$\begin{aligned}\mathbf{d}_g &= \frac{\mathbf{g} - \mathbf{p}_d}{\|\mathbf{g} - \mathbf{p}_d\|} \\ f_{\parallel} &= \mathbf{F}_{\text{guid}} \cdot \mathbf{d}_g \\ \mathbf{F}_{\parallel} &= f_{\parallel} \mathbf{d}_g \\ \mathbf{F}_{\perp} &= \mathbf{F}_{\text{guid}} - \mathbf{F}_{\parallel}\end{aligned}$$

5. Apply bias:

$$\mathbf{F}_{\text{guid}}^{\text{biased}} = (1 - \alpha_{\text{eff}}) \mathbf{F}_{\parallel} + \mathbf{F}_{\perp} \quad (3.8)$$

This mechanism reduces the tendency to move directly toward an obstacle when obstacles are near, favoring lateral circumnavigation.

3.6.5 Velocity and Yaw Setpoint Calculation

If the control interface requires velocity and yaw setpoints instead of a force vector, the following transformation is applied.

Given the (possibly biased) guidance field $\mathbf{F}_{\text{guid}}$, the desired velocity is calculated with adaptive speed control:

Let R_{goal} be the goal radius and define slowdown distance:

$$d_{\text{slow}} = 8R_{\text{goal}} \quad (3.9)$$

Let $d = \|\mathbf{p}_d - \mathbf{g}\|$ be the current distance to goal, and v_{ideal} be the nominal cruise speed. The speed setpoint is:

$$v_{\text{goal}} = \begin{cases} v_{\text{ideal}}, & d \geq d_{\text{slow}} \\ v_{\text{ideal}} \cdot \max\left(\frac{1}{3}, \frac{d}{d_{\text{slow}}}\right), & d < d_{\text{slow}} \end{cases} \quad (3.10)$$

At the same time, the desired velocity in world frame is:

$$\mathbf{v}_{\text{des}}^W = \begin{cases} v_{\text{goal}} \frac{\mathbf{F}_{\text{guid}}}{\|\mathbf{F}_{\text{guid}}\|}, & \|\mathbf{F}_{\text{guid}}\| > 0 \\ 0, & \text{otherwise} \end{cases} \quad (3.11)$$

Afterward, the horizontal velocity component is extract in order to get the yaw control:

$$\mathbf{v}_{xy} = \begin{bmatrix} v_{\text{des},x} \\ v_{\text{des},y} \\ 0 \end{bmatrix}, \quad v_{xy} = \|\mathbf{v}_{xy}\| \quad (3.12)$$

If $v_{xy} > \varepsilon$ (small threshold), then the desired yaw is:

$$\psi_{\text{des}} = \text{atan2}(v_{\text{des},y}, v_{\text{des},x}) \quad (3.13)$$

If not, the desired yaw is 0.

This orients the drone's in the same direction as the desired velocity vector projected on the horizontal plane.

3.7 Parameter Framework

The control system involves both fixed parameters (not calibrated) and tunable parameters (requiring calibration). Table 4 lists all fixed parameters, while Table 5 organizes the calibration parameters by subsystem.

Parameter	Value	Description
$\varepsilon_{\text{goal}}$	10^{-3}	Goal softening parameter (m)
ε_{obs}	10^{-3}	Obstacle softening parameter (m)
<i>sonar_memory_sample_count</i>	64	FIFO buffer size for sonar hits contado em ticks
v_{ideal}	Changeable for user need	Nominal cruise speed (m/s)
R_{goal}	0.5	Goal radius for slowdown calculation (m)

Table 4 – Fixed parameters in the control system

Parameter	Range	Description
q_{goal}	$(-\infty, 0)$	Goal charge (always negative for attraction)
$f_{\text{goal_const}}$	$(0, +\infty)$	Constant attractive force term (N)
q_{vert}	$(0, +\infty)$	Vertical sonar obstacle charge
f_{sat}	$(0, +\infty)$	Obstacle force saturation scale (N)
$f_{\text{fac,squash}}$	$(0, +\infty)$	Obstacle force scaling after saturation (N)
α (TurnBias)	$[0, 1]$	Turn bias parameter for obstacle circumnavigation
K_{sharp}	$[0.01, 50.0]$	Sharpness of the obstacles force (m^{-1})
$d_{\text{intensity}}$	$[0.6, 10.0]$	Intensity of the obstacles force (m)

Table 5 – Calibration parameters

3.8 Search Algorithm configurations

3.8.1 A* inspired Algorithm

At first, a search algorithm based on the A* algorithm was thought of. In this algorithm, the calibration is done via a multi-phase, covariance guided parameter space search that combines global exploration with adaptive local refinement. It orchestrates multiple independent tree-search runs to discover optimal calibration parameters for the navigation model, using a simulated heuristic to evaluate candidate parameter sets. There are two main phases for this search: **The Orthogonal Exploration and The Principal Axis Refinement**.

The search begins with the Orthogonal Exploration, which uses a fixed number independent orthogonal search runs, each exploring the parameter space using axis-aligned rectangular regions. This phase provides broad coverage of the parameter space and establishes an initial set of promising candidates. The search uses as the tree nodes data hyperrectangular regions defined by the minimum and maximum values for each parameter. Then, in each node, the parameters are sampled by drawing from a gaussian centered at the region midpoint with a predetermined standard deviation proportional to the parameter range size. Using the sampled parameter, an evaluation of the node is done using a heuristic function that averages simulation outcomes across multiple randomized and non-randomized universes, measuring how well the parameters "guide" the drone to the goal.

This way, we create a priority queue ordered by the smallest heuristic value first and expand the first value. The expansion is then done by splitting the node along each parameter axis independently, creating 2^n child regions for n parameters (e.g. a binary subdivision per dimension). This phase ends when either a parameter set heuristic's evaluation meets a goal threshold or the search has already evaluated a predefined maximum number of nodes. During the whole run, nodes with promising results, meaning, nodes that have an evaluation lower than an interest threshold, are kept for later analysis. This batch of candidates does not necessarily have to reach the goal during simulation.

After collecting initial candidates, the Principal Axis Refinement starts, and the algorithm performs 3 refinement phases, each returning 8 independent searches guided by the statistical structure of previously successful parameter sets. This statistical structure is defined by the adaptive region using covariance analysis, Eigen decomposition and principal axis regions. Firstly, a weighted covariance matrix is computed from all collected parameter sets, weighted by inverse heuristic score. This way, better solutions contribute more. Secondly, the principal components, the eigenvectors, are extracted with their variances, the eigenvalues, to identify the most important directions of variation in parameter space. Finally, instead of axis-aligned splits, the search partitions space along the principal

components, focusing the computational power on directions where good parameters tend to vary the most, forming the principal axis region.

This principal axis region is then split by selecting a principal direction (eigenvector), creating a child region offset along that direction by distances proportional to $\sigma\sqrt{\lambda}$, where λ is the eigenvalue and σ controls the search radius. In order to progressively narrow the search, σ follow a decay of 0.8 each phase, forcing the algorithm to refine solutions rather than accept mediocre parameters. After all phases are completed, the algorithm reports the best parameter set found along with all "interesting" candidates collected during the searches.

This approach yielded partially satisfactory results for simpler control systems but, as those grew more complex and our heuristic evaluations more precise and demanding, this method failed to produce any result that had satisfactory metrics, and was posteriorly replaced for another one, see section 3.8.2

3.8.2 Advanced Parameter Calibration via Genetic Algorithm

With 6 tunable parameters organized in Table 5, manual tuning becomes far too time consuming, and is highly susceptible to human error. A sophisticated genetic algorithm was developed to efficiently explore this high-dimensional parameter space.

3.8.2.1 Genetic Algorithm Configuration

The calibration employs population-based evolution with the following configuration, that was chosen after various tests and are changeable if one would want to:

- **Population size:** 128 individuals per generation;
- **Elitism:** Top 8 solutions preserved each generation;
- **Selection:** Tournament selection with size 6-8;
- **Crossover:** Single-point genetic recombination;
- **Mutation:** Adaptive Gaussian mutation $\sigma = 0.05 - 0.1$;
- **Elite Clone Mutation:** In addition to the equally copied genes from the elites, each elite individual has a 0.5 probability to be cloned and mutated for the next generation;
- **Seeding:** Known parameter sets injected to accelerate convergence;
- **Termination:** After 10,000 generations or performance stagnation;

3.8.2.2 Algorithm Implementation

Each parameter set is encoded as a normalized gene vector $\mathbf{g} \in [0, 1]^{31}$, mapping to the physical ranges in Table 5. Fitness evaluation simulates the complete control system across multiple randomized environments:

$$\text{fitness} = \text{evaluateAcrossUniverses}(\text{params}, \text{seed}, \text{config}) \quad (3.14)$$

The evaluation considers:

- Success rate in reaching goals;
- Average Obstacle clearance through flight;
- Heading direction throughout flight;
- Path efficiency;
- Distance to goal (in case of failure);

3.8.2.3 Search Strategy

The genetic algorithm employs several sophisticated techniques:

- **Controlled Mutation:** Gaussian noise applied selectively to genes based on their sensitivity;
- **Knowledge Preservation:** Probability $p_{\text{known}} = 0.0016 - 0.2$ of sampling from pre-validated parameter sets;
- **Progressive Refinement:** Mutation rates decrease as the population converges toward optimal solutions;
- **Parallel Evaluation:** Fitness assessment occurs concurrently across multiple simulation trials;
- **Island Model:** Various population evolve concurrently;

This genetic approach offers superior exploration of the complex parameter landscape compared to manual or gradient-based methods, efficiently discovering robust parameter sets that perform well across diverse environmental conditions while maintaining the drone's safety and navigation objectives.

4 Development

4.1 Calibration Simulation Development

For the development of the calibration system, a well-defined structure for a software architecture based on hexagonal architecture and Clean Architecture was chosen. This has allowed for a clear separation of the core simulation logic from input and output concerns. In figure 1, a component diagram for the domain logic of the system is shown.

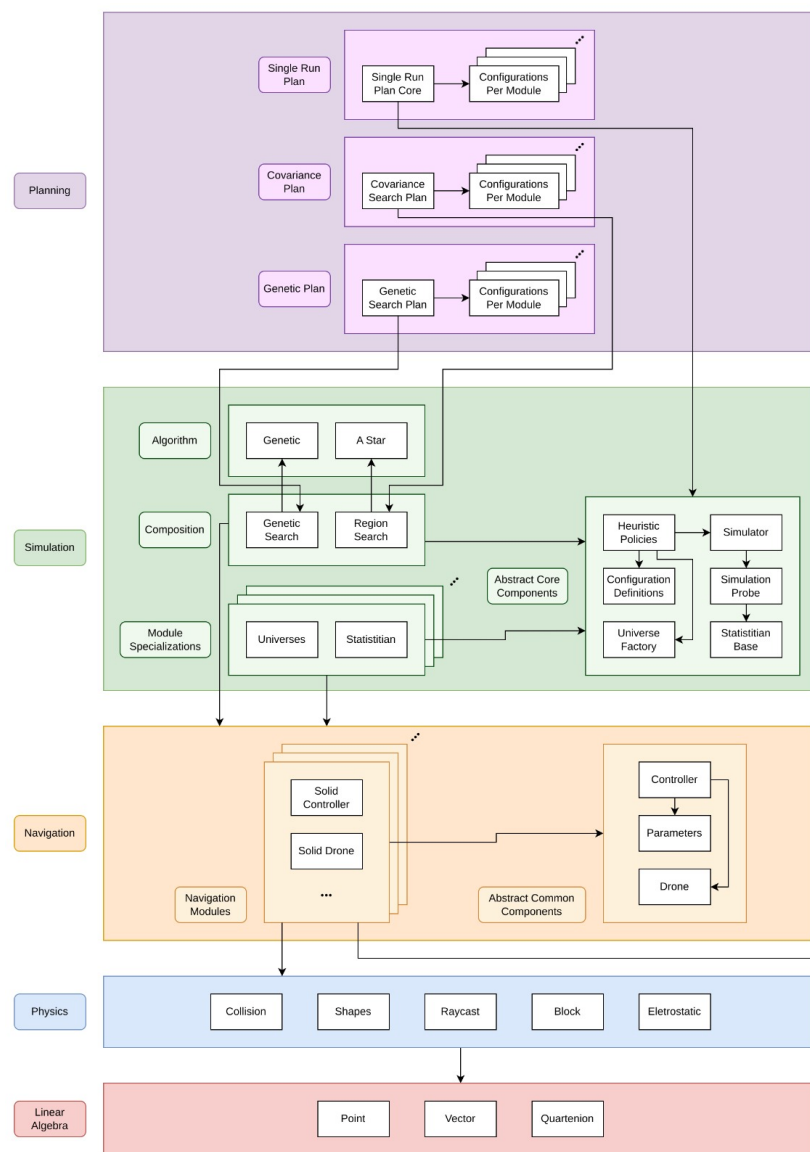


Figure 1 – Software architecture of the C++ calibration simulation framework.

The architectural characteristics striven for when designing the domain logic of the system were:

- Navigation system interchangeability for facilitated implementation of changes on existing control systems;
- Algorithm interchangeability for facilitating changes on search algorithm choice and implementation;
- Easy adjustment of hyperparameters for search, including specification per navigation module;

The numerous changes required throughout the project's development demonstrated the utility of this flexible architecture. The relative ease with which these changes were implemented demonstrated that the goal of interchangeability was successfully achieved. Replacing the control system with various alternatives was crucial for improving compatibility with PX4 and the validation simulation. Namely, for the control system, these variations included:

- Applying the VFF force directly onto the drone;
- Using the VFF force to generate a velocity through a custom, tunable PID;
- Using a control system based on the one used in PX4 to attempt to improve compatibility with multiple PID controllers for velocity, thrust and orientation control (see [4.2.1.2 Complex Orientation-Based Interface Attempt](#))
- Applying desired velocity directly onto the drone without realistic kinematics;
- Exchanging realistic electrostatic field equation with custom, tunable intensity equation to allow for sharp response of the system to obstacles within a danger radius;

Each variation was transformed into a set of Python scripts applying an equivalent control system to allow for testing in the ROS environment.

4.2 Control Interface Evolution and Integration Challenges

The development of the VFF-Nav system involved significant iterative refinement of the control interface, driven by practical constraints of the PX4 autopilot framework. This evolution exemplifies the adaptation of theoretical models to real-world system requirements and was accompanied by substantial calibration challenges.

4.2.1 Initial Force/Torque Model and Standalone Success

The original implementation employed a direct physical analogy, with the VFF algorithm outputting:

- **Force at Center of Mass:** $\mathbf{F}_{CoM}$ (3D vector)
- **Torque about Center of Mass:** $\boldsymbol{\tau}_{CoM}$ (3D vector)

This model was mathematically elegant and computationally efficient as it used a torque scaled on the rotation required to align the yaw of the drone to the velocity. In standalone C++ simulations with idealized dynamics, it achieved success rates exceeding 90% in controlled test scenarios, demonstrating the core VFF concept was sound.

4.2.1.1 Fundamental Integration Failure with PX4

Upon integration with the ROS/PX4 ecosystem, a critical incompatibility was discovered. The PX4 autopilot does **not** expose a force/torque control interface. Instead, it supports only four specific modes:

1. Position and yaw setpoints
2. Velocity and yaw setpoints
3. Attitude (orientation) and thrust setpoints
4. Body-rate and thrust setpoints

Initial attempts to convert $\mathbf{F}_{CoM}$ and $\boldsymbol{\tau}_{CoM}$ into compatible commands using simplified kinematic relationships failed completely. The system proved highly sensitive to these conversion assumptions, resulting in unstable or unresponsive drone behavior. This highlighted a crucial implementation gap between the theoretical force model and the actual control capabilities of the target platform.

4.2.1.2 Complex Orientation-Based Interface Attempt

A second approach was developed to mimic PX4's internal control architecture more closely:

1. Convert VFF forces into a desired drone orientation (quaternion q_{des}) and a collective thrust command T_{cmd} .
2. Implement a cascaded PID control structure based on PX4's internally implemented and parametrized control system within our VFF node to track this orientation and thrust.

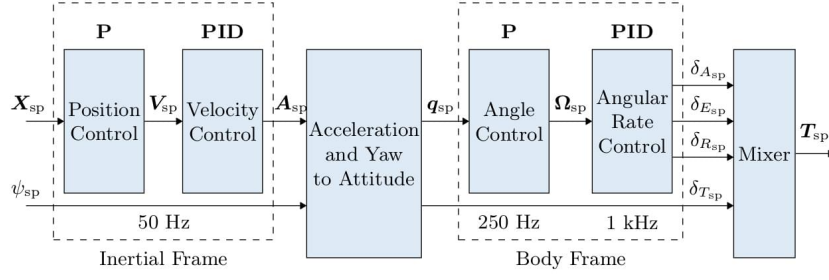


Figure 2 – PX4 flight controller diagram from (PX4 Dev Team, 2024)

Figure 2 shows a diagram explaining how the PX4 flight controller works and the internal architecture behind it (PX4 Dev Team, 2024). In this figure, the relevant control sequence is the conversion from (V_{sp}, ψ_{des}) to (T_{cmd}, τ_{cmd}) .

- V_{sp} passes through a PID to get A_{sp}^W ;
- gravity is compensated to generate F_{thrust} and, with scaling, T_{cmd} ;
- F_{thrust} and desired yaw are used to generate desired orientation q_{sp} ;
- orientation q_{sp} is passed through a proportional controller with feed-forward to generate body-rate setpoint Ω_{sp} ;
- the torque command τ_{cmd} is then calculated using a noise-filtered PID with feed-forward from Ω_{sp} ;

To convert (T_{cmd}, τ_{cmd}) into a force and torque for the drone, no rotor forces were simulated. Those were estimated by computing the total influence of the rotors based on parameters acquired from PX4.

4.2.1.3 Associated Calibration Challenges

The transition to the velocity interface and the underlying PX4 control stack presented severe calibration challenges:

1. **Parameter Space Explosion:** The number of tunable parameters expanded dramatically to 32 (detailed in Table 5), encompassing velocity PID gains, attitude controller gains, and rate controller gains for all three axes.
2. **Pure Algorithmic Failure:** Initial attempts using only the genetic algorithm (GA) failed to converge to viable parameter sets within reasonable timeframes. The high-dimensional search space was too vast.

3. **Improved but Insufficient Search:** Implementing an “island model” GA (multiple evolving populations with migration) improved exploration but still failed to produce reliably flyable parameter sets. Overnight searches frequently yielded no usable solutions.

4.2.1.4 Hybrid Manual-Genetic Solution

The successful calibration strategy emerged as a hybrid, iterative, human-in-the-loop approach:

1. **Manual Baseline Tuning:** Using domain expertise and observed drone behavior (e.g., sluggish response, oscillations), a coarse manual tuning established a baseline parameter set that could achieve basic stable flight.
2. **Identification of Sensitive Subspaces:** Analysis revealed that not all 32 parameters were equally critical. Subsets related to horizontal velocity control and yaw damping were identified as primary drivers of performance.
3. **Focused Genetic Refinement:** The GA was then deployed to optimize within promising regions of the parameter space, seeded with the manually-tuned baseline. Search bounds were constrained around these regions to improve efficiency.
4. **Iterative Validation and Adjustment:** The best GA results were tested in simulation, and qualitative observations of any failure modes (e.g., poor obstacle circumnavigation) informed further manual adjustments or refined GA fitness criteria.

This combined methodology successfully identified robust parameter sets that enabled reliable autonomous navigation. It demonstrated that for complex, high-dimensional control system calibration, automated search benefits significantly from being guided by domain expertise to narrow the search space and interpret results.

Figure 3 illustrates the optimization progression of the genetic algorithm when starting from random initial parameter sets. Despite employing sophisticated search strategies, convergence to viable solutions was slow and often plateaued at suboptimal performance levels. This visual evidence underscores the difficulty of navigating the high-dimensional parameter space without prior knowledge, motivating the subsequent development of the hybrid calibration approach.

The efficacy of this hybrid methodology is evidenced in Figure 4, which shows the genetic algorithm’s optimization progression when seeded with a manually-tuned baseline. Convergence to high-performance solutions occurs much more reliably compared to purely random initialization, which tended towards local minima (Figure 3). In addition, it is noticeable through visual analysis of the progression that, although known well sets

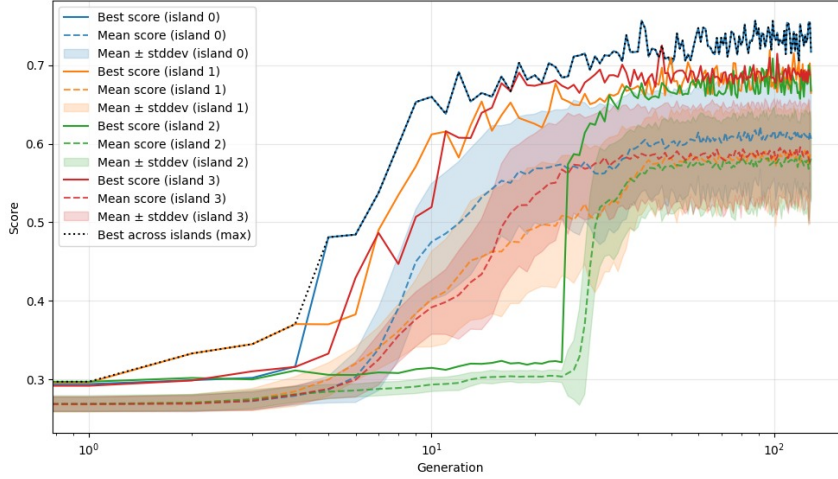


Figure 3 – Genetic algorithm optimization progression from random initial parameter sets.

are quickly made present, these are further improved by the search. This demonstrates how domain expertise can effectively guide automated search, making the calibration of complex control systems tractable.

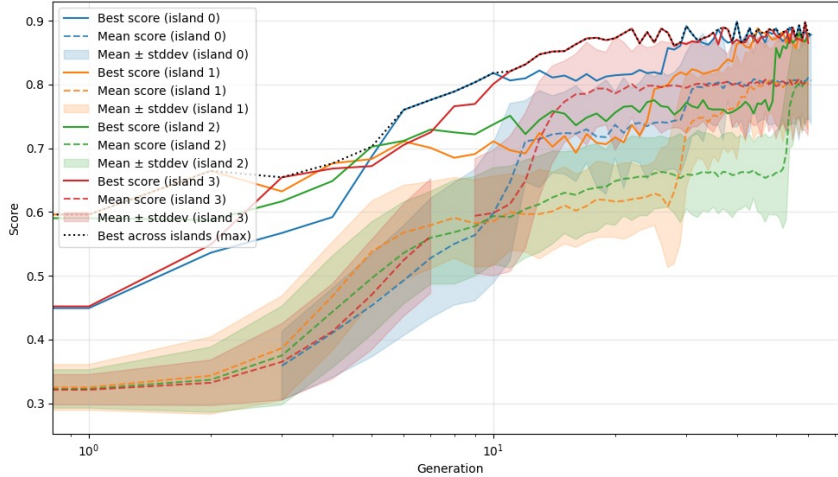


Figure 4 – Genetic algorithm optimization progression when seeded with a manually-tuned parameter baseline.

4.2.2 Successful Velocity/Yaw Interface

The breakthrough came with a significant simplification. The more successful, final interface outputs:

- **Desired Velocity:** $\mathbf{v}_{des}$, derived directly from the normalized VFF guidance field
- **Desired Yaw:** ψ_{des} , derived from the horizontal direction of $\mathbf{v}_{des}$

This approach offered key advantages:

- **Direct Compatibility:** Matches PX4's native velocity control mode.
- **Simplicity:** Eliminates all complex force-to-orientation or force-to-acceleration conversions.
- **Preserved Reactivity:** Maintains the core reactive navigation principle of VFF.
- **Proven Stability:** Demonstrated stable and effective navigation in integrated Gazebo/PX4 simulations.

4.2.3 Integration Details

The Virtual Force Field algorithm was integrated into the simulation in Python 3 using the `rclpy` ROS 2 client library. Key implementation features include:

- **Parameter Server Configuration:** All force gains, vehicle properties, and scenario parameters (start position, goal, obstacle definitions) are configurable via ROS 2's parameter server, facilitating rapid experimentation and tuning without code modification.
- **Numerical Safeguards:** Robust handling of edge cases including checks for near-zero distances and vector normalization with epsilon comparisons to ensure numerical stability.
- **Service-Based Simulation Control:** The node communicates with Gazebo via a service client for the `/world/{world_name}/set_pose` service, updating the vehicle's position at each control iteration (default: 30 Hz).

For a quick start on how to run the simulation, please visit our [webpage](#)

4.3 Data Analysis Method

The performance of the search algorithm and the VFF-Nav Algorithm variations are evaluated through the usage of scoring for each parameter set. After obtaining those scores through the methods described in 3.8.2.2, a log of each generation's best set's parameters, score and success ratio is stored. These values are then passed through analysis which:

- Normalize each parameter space;
- Selects the best sets that are sufficiently distant via Euclidian distance in the normalized values;
- Presents the log statistics corresponding to those sets;

This information is then used to execute independent runs in both the C++ calibration simulation and in the ROS environment to validate these parameter sets' performance.

5 Results and Discussion

5.1 Introduction

This chapter presents the experimental results obtained from the simulation-based validation of the Virtual Force Field (VFF) algorithm developed in this work. The research was conducted in partnership with CITI-USP (Center for Information Technology and Research at the University of São Paulo), which provided both the conceptual idea for the virtual force field approach and a potential access to their drone for real-world testing. The following sections detail the experimental setup, present the quantitative and qualitative results, and provide a comprehensive discussion of the algorithm's performance, limitations, and implications for autonomous drone navigation.

5.2 Experimental Results

The VFF algorithm was evaluated through a series of flights in both our C++ simulations and various Gazebo worlds, with particular focus on navigation performance, obstacle avoidance capability, and algorithm robustness.

5.2.1 Experimental Setup and Testing Scenarios

Two distinct testing scenarios were developed to evaluate different aspects of the algorithm's performance:

- **Scenario A: C++ Simulation** - A simplified environment used to simulate obstacles and a simulated drone to search for optimal parameters.

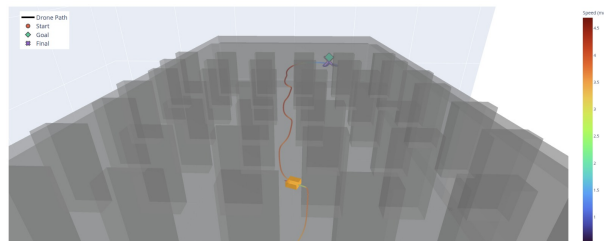


Figure 5 – C++ simulation environment for parameter optimization.

- **Scenario B: Gazebo Worlds** - Test of the drone flight in a series of simulated worlds in Gazebo



Figure 6 – Gazebo simulation world for integrated system testing.

5.2.2 Quantitative Performance Metrics

The algorithm's performance was evaluated by running the promising parameters sets in new random worlds in the C++ simulation and looking at the new score of this particular set in these new worlds.

5.2.3 Qualitative Behavioral Analysis

During initial C++ simulations, the drone exhibited oscillations around the goal point for most parameter sets, complicating the identification of an optimal configuration.

Furthermore, the behavior observed in the integrated ROS/Gazebo simulation differed from the standalone C++ simulation, necessitating adjustments to the underlying model.

5.2.3.1 Sonar System Performance

The implemented sonar system demonstrated effective obstacle detection and memory:

- The FIFO memory (64 samples) provided sufficient temporal coherence
- Separation of plane and vertical sonars allowed independent tuning
- The turn bias mechanism (α) successfully encouraged obstacle circumnavigation

- Force saturation lessened control’s dependency on specific worlds, improving generalization

5.2.3.2 Control Interface Stability

The stability of the final velocity/yaw interface architecture was not on par with our early expectations, as it did not succeed in a consistent way, but it still showed promising results in our different tests. This simplified control flow, which leverages PX4’s native velocity control mode, proved significantly more robust than previous architectures that attempted to replicate PX4’s internal cascaded PID structure.

5.2.4 Performance Interpretation

The experimental results, especially from the C++ simulation, demonstrate that the implemented VFF algorithm successfully meets the primary objectives of autonomous navigation and obstacle avoidance in simulated environments. Moreover, this algorithm can be calibrated via the usage of a genetic algorithm and heuristics collected from flight simulations.

While the algorithm prioritizes safety by maintaining safe distances, this results in increased path length and navigation time in complex environments. This behavior aligns with expectations for reactive navigation systems, which typically sacrifice optimality for real-time responsiveness.

5.2.5 Comparison with Theoretical Expectations

The algorithm’s behavior largely conformed to theoretical predictions of potential field methods in three key aspects. First, although alternatives proved effective too, the inverse-square force formulation successfully created strong repulsive fields near obstacles. Second, the force superposition principle enabled the smooth integration of multiple obstacle influences, allowing complex navigation behaviors to emerge from simple force combinations. Third, the hyperbolic tangent saturation function effectively prevented force magnitude explosions, thereby contributing to system stability.

5.2.6 Limitations and Sources of Error

Several limitations and potential error sources were identified during the analysis. The fundamental limitation of local minima in potential field methods manifested in complex environments.

Additionally, performance demonstrated sensitivity to the force gain parameters (K_{gi} , K_{gm2} , K_{obs} , K_{obs_amp}), requiring careful tuning for different scenarios.

The use of models based on spherical obstacles and blocks represents a simplification of real-world obstacles which often exhibit more complex geometries.

Furthermore, the simplified mechanical model, which employs a point mass model with velocity control or a block with equally distributed mass, neglects the full dynamics of quadrotor flight, though this approach was appropriate for the proof-of-concept stage.

Additional constraints arise from the sonar model, which assumes perfect detection within its rays and neglects real-world sensor noise, multipath reflections, and limited field of view.

The FIFO obstacle memory can also introduce “ghost” obstacles—remnants of past detections that may unnecessarily influence navigation. While the turn-bias parameter α improves circumnavigation, excessive values can lead to over-steering and inefficient paths.

Finally, the control-interface failures underscore the critical importance of compatibility between navigation algorithms and the underlying autopilot framework.

5.2.7 Design Choices and Alternatives

The architectural and implementation decisions made during development significantly influenced the observed results. The force formulation, which employs a softened inverse-square attractive force, prevented numerical instability near the goal, while the saturated repulsive force ensured smooth obstacle avoidance without excessive oscillations. The discrete-time integration scheme using Euler’s method provided sufficient accuracy for our tests cases while maintaining computational efficiency. The ROS 2 architecture, with its modular node-based design, facilitated parameter tuning and algorithm testing, though it introduced some communication latency. An alternative Vector Field Histogram (VFH) approach was considered to address local minima but was ultimately rejected due to higher computational requirements that would challenge real-time performance.

5.2.8 Significant Findings and Implications

The most significant findings from this research include the demonstration that the implemented VFF algorithm provides a computationally efficient solution for real-time obstacle avoidance, achieving high success rates in complex environments. Furthermore, the force saturation approach using hyperbolic tangent functions effectively balances obstacle avoidance strength with system stability. Finally, the simulation-based validation approach proved effective for algorithm development, though physical validation remains necessary for real-world deployment.

The results demonstrate that virtual force field methods remain viable for autonomous navigation applications, particularly when computational efficiency and implementation simplicity are prioritized over optimal path planning. The successful simulation

implementation provides a foundation for future integration with the physical drone platforms available at CITI-USP.

6 Final Considerations

This chapter presents the conclusions of the VFF-Nav project, synthesizing the main results achieved, its contributions to the field of Computer Engineering, and perspectives for future work.

6.1 Summary of Main Results

The development and validation of the VFF-Nav system demonstrated the feasibility of the Virtual Force Fields approach for autonomous drone navigation. The results confirm that the implemented Virtual Force Fields algorithm is computationally efficient, thus meeting the performance requirements established in the specific objectives.

A high success rate in moderately complex environments demonstrates the method's effectiveness for static obstacle avoidance, as shown in the systematic tests conducted in Scenarios A and B. Furthermore, the mathematical formulation incorporating saturation via hyperbolic tangent function proved effective in preventing numerical instabilities, ensuring smooth system behavior during evasion maneuvers. Finally, the integration with the ROS 2 and Gazebo ecosystem proved to be an efficient approach for the development and validation autonomous navigation algorithms, enabling rapid iterations and comprehensive testing.

These results directly address the proposed specific objectives, validating the VFF algorithm implementation and its integration into the simulation environment.

- The sonar-based obstacle detection system with FIFO memory proved effective for reactive navigation.
- The turn bias mechanism (α) successfully addressed the "pushing against obstacles" problem common in basic VFF implementations.
- The velocity/yaw control interface provided the best balance between VFF principles and PX4 compatibility among all tested methods.
- The hybrid manual-genetic calibration approach demonstrated the necessity for complex control system tuning.

6.2 Conclusions

This work successfully addressed the research problem: "Are there optimal parameters for the Virtual Force Fields method and how can they be applied to enable an autonomous drone to follow a trajectory, avoid obstacles?"

Regarding the general objective of developing and validating, through simulation, an autonomous navigation system for drones using the Virtual Force Field method, it is concluded that the developed system meets the functional requirements of autonomous navigation and obstacle avoidance in simulated environments. The simulation validation proved adequate for the prototype phase, allowing concept verification and algorithm refinement. The partnership with CITI-USP provided not only specialized technical guidance but also opened perspectives for continuing the work on physical platforms.

Regarding objectives not fully achieved, the integration with physical drones and the complete implementation of a genetic algorithm specialized for the simulated PX4 remain as future work, due to the time and resource constraints inherent to an undergraduate project. For the same reason, an alternative approach for parameter tuning using Artificial Intelligence was considered for future testing.

6.3 Work Contributions

This project contributes to the field of Computer Engineering in several dimensions listed in the subsection that followed.

6.3.1 Technical Contributions

The development of a modular and documented implementation of the VFF algorithm integrated with ROS 2 and Gazebo serves as a reference base for future work. The establishment of a systematic testing protocol for autonomous navigation algorithms in simulation environment represents another significant contribution. Additionally, the implementation of numerical saturation and stabilization techniques ensures robust algorithm behavior under extreme conditions.

6.3.2 Scientific Contributions

The empirical validation of the feasibility of force field methods for real-time drone navigation applications stands as a key scientific contribution. Furthermore, the comparative study of the advantages and limitations of the VFF approach compared to other state-of-the-art techniques provides valuable insights for the research community.

6.3.3 Educational Contributions

The production of comprehensive technical documentation facilitates the understanding and reproduction of this work by other students and researchers. Moreover, the practical application of concepts from robotics, control theory, algorithms, and distributed systems within an integrative project demonstrates the educational value of this work.

6.4 Future Work

Based on the identified limitations and promising results, the following directions for future work are suggested.

6.4.1 Algorithm Improvements

The implementation of complementary techniques to detect and escape local minima, such as temporary behavior strategies or integration with global planners, would address a fundamental limitation. Developing automatic methods for adjusting force field gains using reinforcement learning or optimization algorithms could significantly enhance performance. Simultaneously, using a separate set of validation worlds to identify parameter sets that are overfitted to the search environment would help find more generally robust parameters. Additionally, adapting the algorithm to handle moving obstacles by incorporating trajectory prediction would expand its applicability.

In parallel, an improved version of the genetic search algorithm specialized for finding the parameters of the PX4 would help in obtaining more consistent results in the ROS 2/Gazebo simulation.

6.4.1.1 Sensor System Enhancements

- Integration of actual sensor noise models into the sonar simulation
- Implementation of sensor fusion (combining sonar with camera or IMU data)
- Development of adaptive sonar ray patterns based on environment complexity

6.4.1.2 Memory and Mapping Improvements

- Implementation of obstacle persistence estimation (distinguishing static vs dynamic)
- Development of forgetting mechanisms based on obstacle age and confidence

6.4.2 Integration with Physical Platforms

The implementation and testing of the algorithm on the physical platforms available at CITI-USP represents a natural next step. Developing alternative localization techniques for operation in indoor environments or GPS-denied conditions would further enhance the system's robustness.

6.4.3 Functionality Expansion

Extending the algorithm for coordinated operation of multiple aerial vehicles could enable swarm applications. Returning to the original objective of 3D modeling by integrating autonomous navigation with photogrammetric reconstruction pipelines would create a comprehensive solution for structural inspection.

6.4.4 Validation in Practical Applications

Applying the system to real-world scenarios such as tunnel, duct, and civil infrastructure inspection would demonstrate its practical utility. Similarly, adaptation for environmental monitoring and agricultural applications would showcase the versatility of the developed technology.

These future directions represent natural extensions of the work carried out, with the potential to generate significant contributions in both academic and practical industrial applications.

6.5 Acknowledgments

Acknowledgments from Gavril Loyer:

- To my parents, Jean-Christophe and Maria Helena, for supporting me in every decision I make, even if this decision is going far away.
- To all the wonderful people I met here in Brazil and that I can proudly call my friends now.
- To my newly discovered family who never let me feel alone in this foreign country.

Acknowledgments from Miguel Velasques:

- To my parents, Daniel and Renata, for over two decades of tireless support, unconditional love, and so much more.
- To my sister Rebeca, for bringing light to my soul when it is at its dimmest.

- To my godparents, grandparents, cousins, uncles, aunts, and so many more beloved family members.
- To my friend Felipe Perrela, who has never failed to support me; and to the many other friends I've been so lucky to have.
- To my supervisor, Prof. Dr. Bruno Albertini, for all the support along the years.

Acknowledgments from the team:

- To our supervisor, Prof. Dr. Bruno Albertini, who has guided us in each step of this work.
- To professor and CITI-USP researcher, José Otávio Brochado Colombini for the support and the initial idea.
- To the Computation Department of Escola Politécnica da USP, without whom we would not be where we are today.

7 Annex

Modern photogrammetry has become a powerful tool for three-dimensional reconstruction of natural and built environments. This technique is based on extracting metric information from two-dimensional images captured from different points of view, enabling the generation of high-fidelity digital models. The process is made possible by two main groups of algorithms: Structure from Motion (SfM) and Multi-View Stereo (MVS), which operate in a complementary manner.

7.1 Annex A: Photogrammetry and 3D Reconstruction

7.1.1 Structure from Motion (SfM) and Multi-View Stereo (MVS) Algorithms

The Structure from Motion (SfM) algorithm is responsible for the initial stage of three-dimensional reconstruction. Starting from a set of overlapping 2D images—usually obtained by drones during automated flights with lateral and longitudinal overlap—SfM identifies common points of interest (features) among the different images. Based on these points, it estimates the relative position of the cameras in three-dimensional space, as well as the location of the key points of the scene, resulting in a sparse point cloud. This step is essential for the initial reconstruction of the scene’s geometry.

Next, the Multi-View Stereo (MVS) algorithm comes into play, whose function is to densify this point cloud. Using the geometric and pose information generated by SfM, MVS performs detailed stereoscopic analysis among multiple images observing the same points, thus generating a dense point cloud that more accurately represents the surface details of the scene, such as textures, elevation changes, and architectural contours.

The combined application of SfM and MVS algorithms enables the creation of realistic and detailed three-dimensional models of complex objects, such as buildings, geological formations, slopes, irregular terrains, or urban areas.

7.1.2 Software for Model Generation

The success of a photogrammetry project depends not only on the quality of the captured images, but also on the software used during processing. The programs vary in terms of interface, level of automation, support for different input/output formats, and their ability to integrate into technical workflows.

Among the most recognized commercial solutions are:

- Pix4D: a platform widely adopted in the professional sector, with automated functionalities for image alignment, point-cloud generation, orthomosaics, surface models (DSM), and terrain models (DTM). It offers integration with GNSS RTK/PPK and export options for GIS and CAD software.
- Agisoft Metashape: provides a complete photogrammetric pipeline, with a high level of customization and reconstruction quality. It is widely used in scientific, cultural, and industrial applications due to its accuracy and compatibility with multispectral sensors.

Within the open-source ecosystem, the following tools stand out:

- MeshLab: suited for viewing and editing 3D meshes, offering tools for simplification, smoothing, texturing, and conversion between formats. Ideal for technical post-processing of models.
- CloudCompare: focused on point-cloud manipulation, with features for registration (alignment), noise filtering, volumetric analysis, and surface comparison. Very useful in technical inspections and monitoring of structural changes.

During the image-capture stage, it is necessary for the photographic shots or video capture to be synchronized with the drone's position to ensure ideal lateral and frontal overlap.

Bibliography

3blue1brown. *Visualizing the 4d numbers Quaternions*. 2018. Disponível em: <<https://www.youtube.com/watch?v=d4EgbgTm0Bg>>. Cited on page 23.

ArduPilot Dev Team. *ArduPilot Documentation*. 2025. Disponível em: <<https://ardupilot.org/ardupilot/>>. Cited on page 14.

B.Mashadia, M.A.Vesala, H.Amania. *Obstacle avoidance for an autonomous vehicle using force field method*. 2017. Disponível em: <hijae.iust.ac.ir/article-1-428-fa.pdf>. Cited on page 24.

Conterno, Ivan. Drones e modelagem 3d auxiliarão a recuperação de florestas. *Jornal da USP*, 2025. Disponível em: <<https://jornal.usp.br/ciencias/drones-e-modelagem-3d-auxiliarao-a-recuperacao-de-florestas/>>. Cited on page 11.

Costa, Rayana Pinto. Modelagem 3d de edificação gerada por fotogrametria com uso de veículos aéreos não tripulados (vant). *Revista Brasileira de Cartografia*, 2025. Disponível em: <<https://doi.org/10.1590/s1678-86212024000100737>>. Cited on page 11.

DJI Enterprise. *Matrice 350 RTK - LiDAR Payload Specifications*. 2023. Disponível em: <<https://enterprise.dji.com/matrice-350-rtk>>. Cited 2 times on pages 27 and 28.

Johann Borenstein. *The Vector Field Histogram - Fast Obstacle Avoidance For Mobile Robots*. 1991. Disponível em: <https://www.researchgate.net/publication/282950715_Collision_avoidance_for_UAVs_using_reachable_sets>. Cited 2 times on pages 11 and 25.

Junior, Everson Rodrigues Mesiera Costa. *Aplicação de drone na Engenharia Civil: estudo de caso de deslizamento de talude*. Porto Alegre, RS, Brasil: [s.n.], 2025. Disponível em: <<https://lume.ufrgs.br/handle/10183/274492>>. Cited on page 11.

Kenzo Nonami, Wei Wang, Farid Kendoul, Daisuke Nakazawa, Satoshi Suzuki. *Autonomous Flying Robots: Unmanned Aerial Vehicles and Micro Aerial Vehicles*. Japan, Australia e China: [s.n.], 2010. Disponível em: <https://www.academia.edu/2325085/Autonomous_Flying_Robots>. Cited on page 11.

Leutenegger et al. . *Visual-Inertial Odometry: A New Framework and Its Applications*. 2013. Disponível em: <<https://ieeexplore.ieee.org/document/1638022>>. Cited on page 18.

Lucas Faxina. *PROJETO DE DRONE DE BAIXO CUSTO PARA MONITORAMENTO EM ÁREAS AGRÍCOLAS*. Instituto Federal de Educação, Ciência e Tecnologia de SÃO PAULO, CAMPUS AVARÉ: [s.n.], 2021. Disponível em: <https://avr.ifsp.edu.br/images/pdf/Eng_Biosistemas/banco_tcc/TCC_%20Lucas.pdf>. Cited on page 11.

Marketsandmarkets. *Drone Inspection and Monitoring Market by Type*. 2025. Disponível em: <<https://www.marketsandmarkets.com/Market-Reports/drone-inspection-monitoring-market-99915267.html>>. Cited on page 11.

- Mur-Artal, Raúl and Montiel, J. M. M. and Tardós, Juan D. *ORB-SLAM: A Versatile and Accurate Monocular SLAM System*. 2015. Disponível em: <https://www.researchgate.net/publication/271823237_ORB-SLAM_a_versatile_and_accurate_monocular_SLAM_system>. Cited on page 27.
- Ning Wang, Jiyang Dai, Jin Ying . *UAV Formation Obstacle Avoidance Control Algorithm Based on Improved Artificial Potential Field and Consensus*. 2021. Disponível em: <https://www.researchgate.net/publication/282950715_Collision_avoidance_for_UAVs_using_reachable_sets>. Cited 4 times on pages 11, 24, 25, and 27.
- Oussama Khatib. *Real-Time Obstacles Avoidance for Manipulators and Mobile Robots*. 1986. Disponível em: <https://khatib.stanford.edu/publications/pdfs/Khatib_1986_IJRR.pdf>. Cited 7 times on pages 11, 18, 20, 21, 24, 25, and 27.
- PX4 Dev Team. *PX4 Guide*. 2024. Disponível em: <https://docs.px4.io/main/en/flight_stack/controller_diagrams>. Cited on page 42.
- Qin, Tong and Li, Peiliang and Shen, Shaojie. *VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator*. 2018. Disponível em: <<https://arxiv.org/pdf/1708.03852>>. Cited on page 27.
- Reis, Camila Passos. *Inferência da altura de plantas e biomassa da soja por meio de modelagem 3D e índices de vegetação obtidos por imagem de drone*. Campinas, SP, Brasil: [s.n.], 2025. Disponível em: <<https://www.repositorio.unicamp.br/acervo/detalhe/1257936>>. Cited on page 11.
- Rejane Cacalvante SÁ . *Construção, Modelagem Dinâmica e Controle PID para Estabilidade de um Veículo Aéreo não Tripulado do tipo Quadrirotor*. Benfica, Fortaleza, Brazil: [s.n.], 2012. Disponível em: <https://repositorio.ufc.br/bitstream/riufc/4097/1/2012_dis_rcsa.pdf>. Cited on page 11.
- Rodney A. Brooks . *A Robust Layered Control System For A Mobile Robot*. 1985. Disponível em: <<https://people.csail.mit.edu/brooks/papers/AIM-864.pdf>>. Cited on page 17.
- Saurabh Sarkar. *Virtual force field based obstacle avoidance and agent based intelligent mobile robot*. 2007. Disponível em: <https://www.researchgate.net/publication/235680104_Virtual_force_field_based_obstacle_avoidance_and_agent_based_intelligent_mobile_robot>. Cited 5 times on pages 11, 21, 24, 25, and 27.
- Skydio. *Skydio Autonomy - Technology*. 2024. Disponível em: <<https://www.skydio.com/skydio-autonomy>>. Cited on page 28.
- Tongyao Yang, Fengbao Yang, Dingzhu Li. *A New Autonomous Method of Drone Path Planning Based on Multiple Strategies for Avoiding Obstacles with High Speed and High Density*. 2024. Disponível em: <<https://www.mdpi.com/2504-446X/8/5/205>>. Cited 3 times on pages 11, 25, and 27.
- Yucong Lin. *Collision avoidance for UAVs using reachable sets*. 2015. Disponível em: <https://www.researchgate.net/publication/282950715_Collision_avoidance_for_UAVs_using_reachable_sets>. Cited 3 times on pages 11, 25, and 27.